

Authentication Flow

Gema Event Management Platform
Generated: 2026-02-25

1. Registration Flow

```
sequenceDiagram
    participant C as Client
    participant RL as Rate Limiter (5/15min)
    participant V as Validator
    participant AC as AuthController
    participant DB as MongoDB
    participant E as EmailQueue (BullMQ)

    C->>RL: POST /api/auth/register
    RL-->>C: 429 if exceeded
    RL->>V: validateRegistration
    V-->>C: 400 if invalid
    V->>AC: register()
    AC->>DB: User.findOne({email})
    DB-->>AC: existing user?
    AC-->>C: 400 Email already registered
    AC->>DB: bcrypt.hash(password) + User.create()
    AC->>E: Queue: send verification email (OTP)
    AC->>DB: RefreshToken.create()
    AC-->>C: 201 + Set-Cookie: accessToken (httpOnly) + refreshToken
```

2. Login Flow

```
sequenceDiagram
    participant C as Client
    participant RL as Rate Limiter (5/15min, skip success)
    participant V as Validator
    participant AC as AuthController
    participant Redis as Redis Cache
    participant DB as MongoDB

    C->>RL: POST /api/auth/login
    RL-->>C: 429 Too many login attempts
    RL->>V: validateLogin (email, password)
    V-->>C: 400 if invalid
    V->>AC: login()
    AC->>DB: User.findOne({email}).select(+password)
    DB-->>AC: user not found → 401
    AC->>AC: bcrypt.compare(password, hash)
    AC-->>C: 401 Invalid credentials
    AC->>AC: jwt.sign({id}, secret, {expiresIn})
    AC->>DB: RefreshToken.create()
    AC->>Redis: SET user:{id} (TTL 600s)
    AC-->>C: 200 + Set-Cookie: accessToken + refreshToken (httpOnly)
```

3. Request Authentication Flow

```
sequenceDiagram
    participant C as Client
    participant AM as authenticate() middleware
    participant Redis as Redis Cache (TTL 600s)
    participant DB as MongoDB
    participant Ctrl as Controller

    C->>AM: Request with cookie/header
    AM->>AM: Extract token (cookie first, then Bearer header)
    AM-->>C: 401 Not authorized (no token)
    AM->>AM: jwt.verify(token, secret, {clockTolerance: 10s})
    AM-->>C: 401 Token expired / Invalid token
    AM->>Redis: GET user:{decoded.id}
    alt Cache hit
        Redis-->>AM: user object
    else Cache miss
        AM->>DB: User.findById(decoded.id)
        DB-->>AM: 404 User not found
        AM->>Redis: SET user:{id} TTL=600s
    end
    AM->>Ctrl: next() with req.user attached
```

4. Firebase Authentication Flow

```
sequenceDiagram
    participant C as Client (Mobile/OAuth)
    participant RL as Rate Limiter
    participant V as Validator
    participant AC as AuthController
    participant FB as Firebase Admin SDK
    participant DB as MongoDB

    C->>RL: POST /api/auth/firebase {idToken}
    RL->>V: validateFirebaseAuth
    V->>AC: firebaseAuth()
    AC->>FB: getAuth().verifyIdToken(idToken)
    FB-->>AC: 401 Firebase authentication failed
    AC->>DB: User.findOne({email: decodedToken.email})
    alt User exists
        AC->>AC: Generate JWT
    else New user
        AC->>DB: User.create({email, name, avatar, firebaseUid})
        AC->>AC: Generate JWT
    end
    AC-->>C: 200 + Set-Cookie: accessToken (httpOnly)
```

5. Token Refresh Flow

```
sequenceDiagram
    participant C as Client
    participant AC as AuthController
    participant DB as MongoDB
    participant Redis as Redis

    C->>AC: POST /api/auth/refresh-token {refreshToken}
    AC->>DB: RefreshToken.findOne({token, userId})
    DB-->>AC: 401 Invalid or expired refresh token
    AC->>AC: Verify refresh token not blacklisted
    AC->>AC: jwt.sign({id}, secret) - new access token
    AC->>DB: RefreshToken.rotate() - invalidate old, create new
    AC->>Redis: DELETE user:{id} - invalidate cache
    AC-->>C: 200 + Set-Cookie: accessToken (new, httpOnly)
```

6. Password Reset Flow

```
sequenceDiagram
    participant C as Client
    participant RL as Rate Limiter (3/hour)
    participant AC as AuthController
    participant DB as MongoDB
    participant E as EmailQueue

    C->>RL: POST /api/auth/forgot-password {email}
    RL-->>C: 429 Too many password reset requests
    AC->>DB: User.findOne({email})
    Note over AC: Always return 200 (prevent email enumeration)
    AC->>DB: Generate reset OTP, store with expiry
    AC->>E: Queue: send password reset email
    AC-->>C: 200 (regardless of whether email exists)

    C->>RL: POST /api/auth/reset-password {token, newPassword}
    RL-->>C: 429
    AC->>DB: Find user by reset token + check expiry
    DB-->>AC: 400 Invalid or expired token
    AC->>DB: bcrypt.hash(newPassword) + clear reset token
    AC-->>C: 200 Password reset successful
```

7. Email Verification Flow

```
sequenceDiagram
    participant C as Client
    participant RL as Rate Limiter (5/hour)
    participant AC as AuthController
    participant DB as MongoDB

    C->>RL: POST /api/auth/verify-email {otp}
    RL-->>C: 429
    AC->>DB: User.findOne({emailVerificationOTP: otp})
    DB-->>AC: 400 Invalid OTP
    AC->>DB: Check OTP expiry
    AC-->>C: 400 OTP expired
    AC->>DB: User.update({isEmailVerified: true, clear OTP})
    AC-->>C: 200 Email verified

    Note over C,DB: Resend: POST /api/auth/resend-verification-email (5/hour limit)
```

8. Phone Verification Flow

```
sequenceDiagram
    participant C as Authenticated Client
    participant AM as authenticate()
    participant RL as Rate Limiter (5/hour)
    participant AC as AuthController
    participant SMS as SMSService

    C->>AM: POST /api/auth/send-phone-verification {phone}
    AM->>AC: (authenticated)
    AC->>SMS: Send OTP via SMS
    AC-->>C: 200 OTP sent

    C->>AM: POST /api/auth/verify-phone {otp}
    AM->>AC: (authenticated)
    AC->>DB: Verify OTP + expiry
    AC->>DB: User.update({isPhoneVerified: true})
    AC-->>C: 200 Phone verified
```

9. Authorization Middleware

flowchart TD

```
A[Incoming Request] --> B{authenticate()}
B -->|No token| C[401 Not authorized]
B -->|Token invalid| D[401 Invalid token]
B -->|Token expired| E{How old?}
E -->|>1 day| F[401 Expired N days ago, re-login]
E -->|<1 day| G[401 Use /refresh-token]
B -->|Token valid| H[req.user attached]
H --> I{authorize(roles)?}
I -->|No role requirement| J[Controller]
I -->|Role matches| J
I -->|Role mismatch| K[403 Not authorized for this action]
K --> L[Log warn: endpoint + userRole + userId]
```

10. API Endpoints Summary

Method	Endpoint	Auth	Rate Limit
POST	/api/auth/register	Public	authLimiter (5/15min)
POST	/api/auth/login	Public	authLimiter (5/15min, skip success)
POST	/api/auth/logout	Public	none
POST	/api/auth/refresh-token	Public	none
GET	/api/auth/me	Optional	none
GET	/api/auth/profile	Required	none
PUT	/api/auth/profile	Required	none
PUT	/api/auth/change-password	Required	none
POST	/api/auth/forgot-password	Public	passwordResetLimiter (3/hour)
POST	/api/auth/reset-password	Public	passwordResetLimiter (3/hour)
POST	/api/auth/verify-email	Public	emailVerificationLimiter (5/hour)
POST	/api/auth/resend-verification-email	Public	emailVerificationLimiter (5/hour)
POST	/api/auth/firebase	Public	authLimiter (5/15min)
POST	/api/auth/addresses	Required	none
PUT	/api/auth/addresses/:index	Required	none
DELETE	/api/auth/addresses/:index	Required	none
POST	/api/auth/send-phone-verification	Required	emailVerificationLimiter
POST	/api/auth/verify-phone	Required	emailVerificationLimiter
POST	/api/auth/resend-phone-verification	Required	emailVerificationLimiter